



University of New York Tirana

Algorithms & Complexity

Host Connectivity in a Data Farm

Technical Report by Frenki Selmani

Tirana, 2026

1. The problem statement

The goal of this assignment is to analyze a hypothetical data farm made from computational hosts and directional links. Each host is represented by an integer from 0 to $n - 1$.

Each directed link has a non-negative latency value, that is measured in nanoseconds. Starting from one given host, the program must compute the fastest possible transfer chain from that starting host to every other host.

A transfer chain is a sequence of valid directed links. The total latency of a chain is the sum of the latencies of all links in that chain. The empty chain has latency 0, which applies to the starting host itself.

There is also an optional link budget. If the budget is -1 , chains may have any number of links. If the budget is a non-negative integer, then only chains using at most that many links are acceptable.

The problem is therefore a shortest path problem on a directed weighted graph.

The additional budget condition creates a second version of the problem: shortest paths with a maximum number of allowed edges.

2. Input structure

The input file has the following structure:

n e

budget

start

fromHost toHost weight

fromHost toHost weight

Here, n is the number of hosts, e is the number of directed links, budget is the maximum allowed number of links, and start is the starting host. Each following line describes a directed link from one host to another with a given latency. Here is the example input used for testing:

5 9

-1

0

0 1 4

0 2 2

1 2 3

2 1 1

1 3 2

1 4 3

2 3 4

2 4 5

4 3 1

3. My approach

The solution separates the problem into two cases. This is important because the best algorithm depends on whether the budget exists or not.

Case 1: If $\text{budget} = -1$, there is no limit on the number of links. Since all link weights are non-negative, Dijkstra's algorithm is used. Dijkstra's algorithm is appropriate because it efficiently computes the shortest path from one source node to all other nodes in a graph with non-negative edge weights.

Case 2: If $\text{budget} \geq 0$, the number of links is limited. In this case, normal Dijkstra's algorithm alone is not enough because the number of links used in the path matters. The solution uses a dynamic-programming style relaxation algorithm. It computes the best distance to each host using exactly 1 link, exactly 2 links, and so on up to the budget. The final answer is the best distance found using at most the budget number of links.

4. Algorithms that were used

Dijkstra Algorithm for the No-Budget Case

When the budget is -1, the program initializes the distance of every host as infinity, except the starting host, which receives distance 0. A priority queue is used to always process the currently known closest host first. For every outgoing link from that host, the algorithm checks whether using that link creates a faster route to the neighboring host. If a faster route is found, the distance and parent of the neighboring host are updated.

```
if budget == -1:
    distances, paths = dijkstra(n, graph, start)
for neighbor, weight in graph[current_node]:
    new_dist = current_dist + weight

    if new_dist < dist[neighbor]:
        dist[neighbor] = new_dist
        parent[neighbor] = current_node
        heapq.heappush(priority_queue, (new_dist, neighbor))
```

Budget-Limited Shortest Path Algorithm

When the budget is limited, the algorithm uses repeated edge relaxation. The main idea is to keep track of the best latency to reach each host using exactly a certain number of links. For every step from 1 to the effective budget, the program checks all edges. If host u can be reached using $step - 1$ links, then an edge $u \rightarrow v$ can be used to reach v using $step$ links.

```
candidate = exact_prev[u] + weight
if candidate < exact_current[v]:
    exact_current[v] = candidate
    parent_state[(step, v)] = u
```

The array `best_dist` stores the best latency found so far using at most the allowed number of links. This makes the final output satisfy the budget constraint.

5. Data structures

The implementation uses the following data structures:

Adjacency list: `graph[u]` stores all outgoing links from host `u`. This is efficient for Dijkstra because it quickly finds all neighbors of the current host.

Edge list: `edges` stores all directed links as triples `(u, v, weight)`. This is useful for the budget-limited algorithm, which repeatedly scans all edges.

Priority queue: Python `heapq` is used in Dijkstra to always process the smallest current distance first.

Parent arrays/dictionaries: `parent` and `parent_state` store previous hosts so that the actual path can be reconstructed, not only the latency.

```
graph[from_host].append((to_host, weight))
edges.append((from_host, to_host, weight))
```

6. Sub-problems

The assignment can be divided into smaller sub-problems. First, the input must be read and converted into a graph representation. Second, the program must decide whether the instance has a link budget or not. Third, it must compute the minimum latencies using the appropriate algorithm. Fourth, it must reconstruct the transfer chains from the stored parent information. Finally, it must print clear results for every host.

7. Time complexity analysis

Dijkstra Case

Let n be the number of hosts and e be the number of links. With an adjacency list and a priority queue, the time complexity of Dijkstra's algorithm is $O((n + e) \log n)$. The space complexity is $O(n + e)$, because the graph stores all vertices and edges, and the algorithm also stores distance, parent, and priority queue information.

Budget-Limited Case

Let B be the effective budget. The algorithm runs one relaxation phase for each possible number of links from 1 to B . In each phase, it scans all e edges. Therefore, the main running time is $O(B * e)$. The implementation also scans the n hosts after each step to update `best_dist`, giving $O(B * (e + n))$ overall.

The effective budget is limited to $n - 1$, because with non-negative weights, a shortest useful path does not need cycles. Cycles can be removed without increasing latency. The space complexity is $O(n + Bn)$ in the conceptual dynamic programming view. The implementation stores current and previous distance arrays, plus parent information for path reconstruction.

8. Output Format

Since the assignment did not specify a required output format, the implementation uses a simple readable format. For each host, the program prints either unreachable or the minimum latency and the corresponding path.

Host 3: latency = 5, path = 0 -> 2 -> 1 -> 3

Host 5: unreachable

9. Testing

Test 1: No Budget

For the sample input with budget -1, the program produces:

Starting host: 0

Budget: No limit

Host 0: latency = 0, path = 0

Host 1: latency = 3, path = 0 -> 2 -> 1

Host 2: latency = 2, path = 0 -> 2

Host 3: latency = 5, path = 0 -> 2 -> 1 -> 3

Host 4: latency = 6, path = 0 -> 2 -> 1 -> 4

Test 2: Budget = 2

Using the same graph but setting the budget to 2 gives:

Starting host: 0

Budget: At most 2 links

Host 0: latency = 0, path = 0

Host 1: latency = 3, path = 0 -> 2 -> 1

Host 2: latency = 2, path = 0 -> 2

Host 3: latency = 6, path = 0 -> 1 -> 3

Host 4: latency = 7, path = 0 -> 1 -> 4

This confirms that the budget condition changes the result. Without a budget, the best path to host 3 has latency 5 using three links. With budget = 2, that path is not allowed, so the best allowed latency becomes 6.

10. Important bits of code

Algorithm Selection

```
233
234     if budget == -1:
235         print("Budget: No limit")
236     else:
237         print(f"Budget: At most {budget} links")
```

This part of the code checks whether the input has a link budget or not. If the budget is -1, the program uses Dijkstra's algorithm; otherwise, it uses the budget-limited shortest path algorithm.

Dijkstra's Algorithm

```
99         for neighbor, weight in graph[current_node]:
100             new_dist = current_dist + weight
101
102             if new_dist < dist[neighbor]:
103                 dist[neighbor] = new_dist
104                 parent[neighbor] = current_node
105                 heapq.heappush(priority_queue, (new_dist, neighbor))
```

This part of the code checks whether the input has a link budget or not. If the budget is -1, the program uses Dijkstra's algorithm; otherwise, it uses the budget-limited shortest path algorithm.

Budget-Limited Shortest Path Algorithm

```
189         for step in range(1, effective_budget + 1):
190             exact_current = [INF] * n
191             changed = False
192
193             for u, v, weight in edges:
194                 if exact_prev[u] == INF:
195                     continue
196
197                 candidate = exact_prev[u] + weight
198
199                 if candidate < exact_current[v]:
200                     exact_current[v] = candidate
201                     parent_state[(step, v)] = u
```

This part of the code checks whether the input has a link budget or not. If the budget is -1, the program uses Dijkstra's algorithm; otherwise, it uses the budget-limited shortest path algorithm.

11. Bonus

For the bonus task, the program computes the longest simple chain from the starting host to every other host. The implementation first checks whether the graph is acyclic and, if so, uses topological ordering and dynamic programming to efficiently compute the longest paths. For small graphs that contain cycles, the program uses DFS to find the exact longest simple chains while ensuring that no host is visited more than once.

```
338  def longest_simple_chain_small_graph(n, graph, start):
339
340      longest = [-INF] * n
341      best_paths = [None] * n
342
343      visited = [False] * n
344      current_path = []
345
346      def dfs(node):
347          visited[node] = True
348          current_path.append(node)
349
350          chain_length = len(current_path) - 1
351
352          if chain_length > longest[node]:
353              longest[node] = chain_length
354              best_paths[node] = current_path.copy()
355
356          for neighbor, weight in graph[node]:
357              if not visited[neighbor]:
358                  dfs(neighbor)
359
360          current_path.pop()
361          visited[node] = False
```

This function computes the longest simple chain using DFS and backtracking. The visited array prevents cycles by ensuring that each host appears at most once in a path, while the algorithm keeps track of the longest valid chain found for each host.